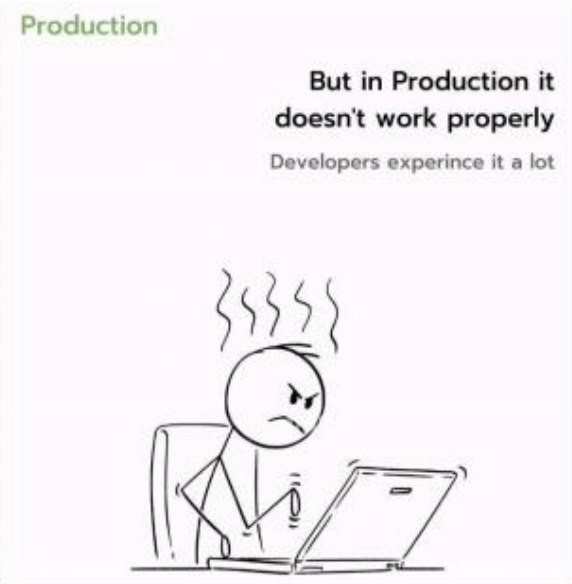
Docker

Why docker?

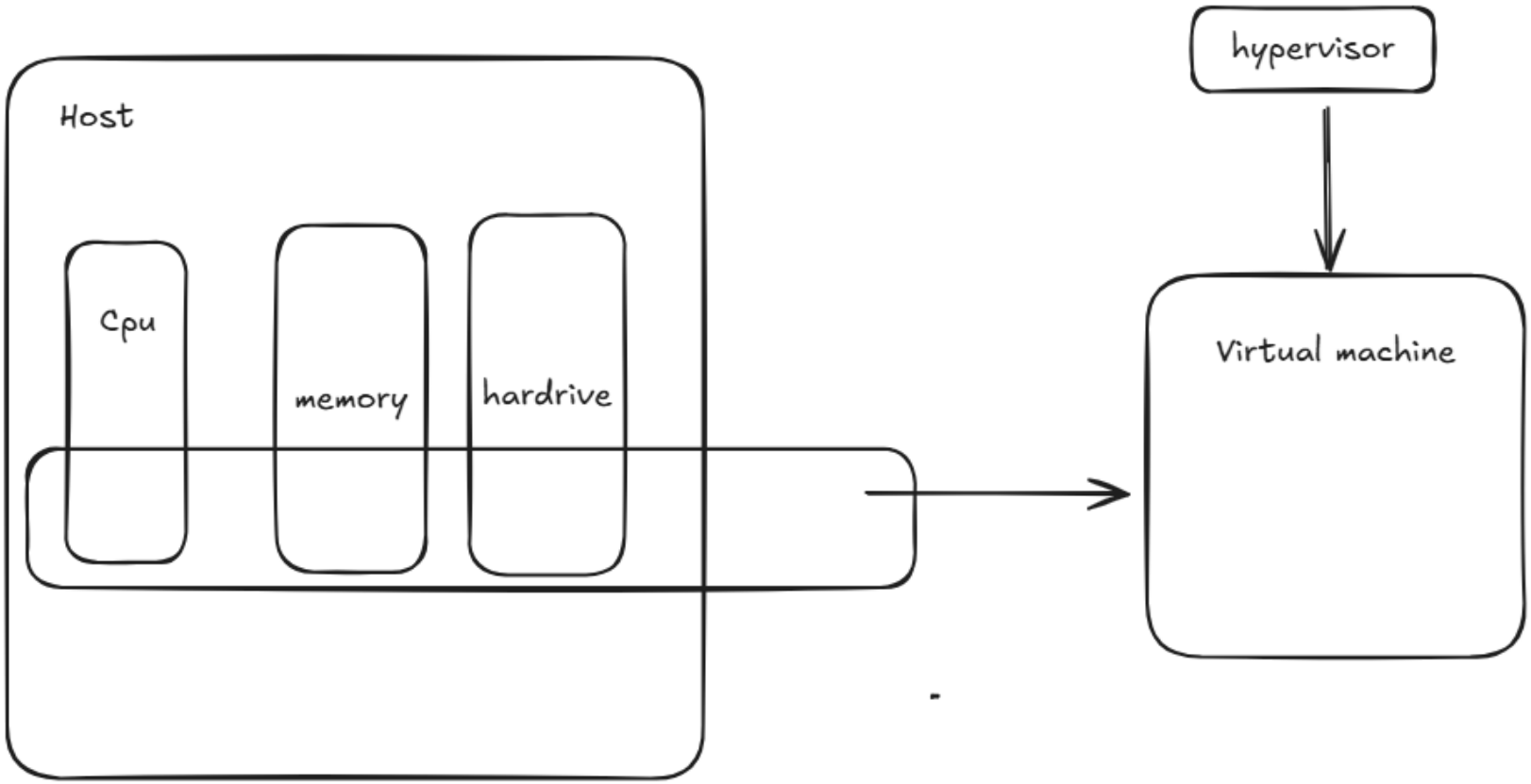
Motivation

Development
Lets say You created an Application
And that's working fine in your machine

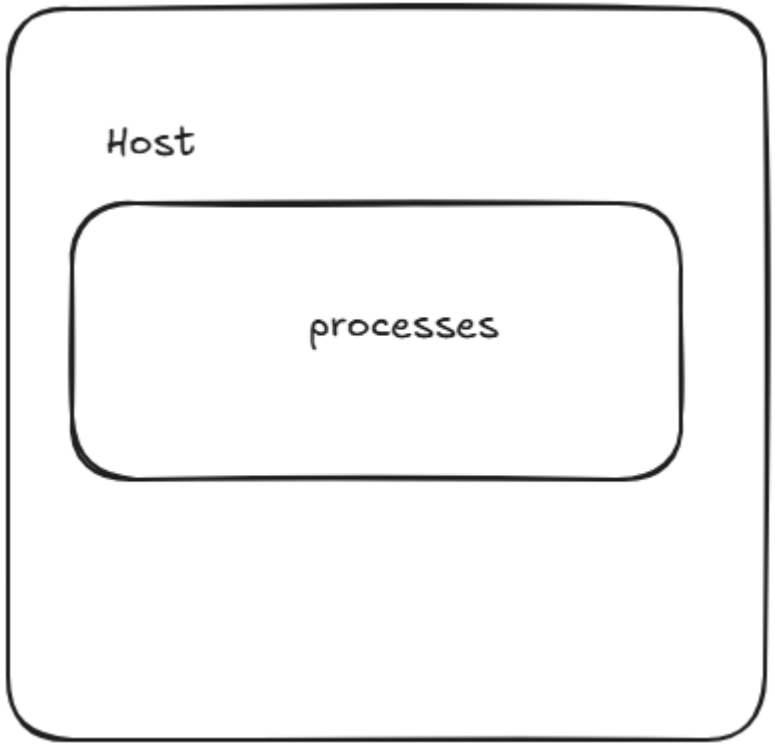

Production
But in Production it doesn't work properly
Developers experince it a lot

That is when the Developer's famous words are spoken
It works on my machine
Your application is not working

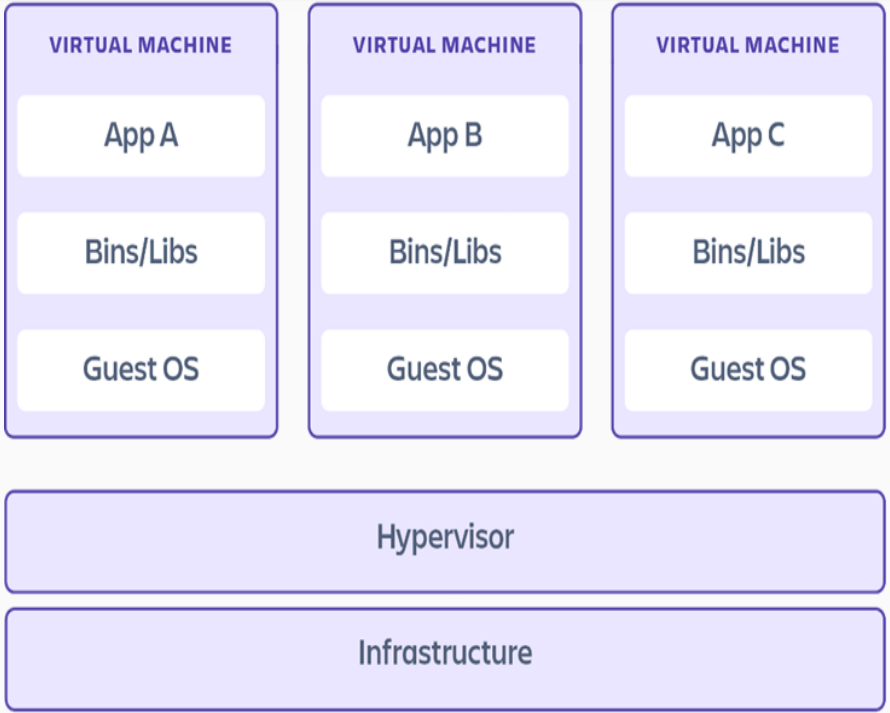
Virtualization

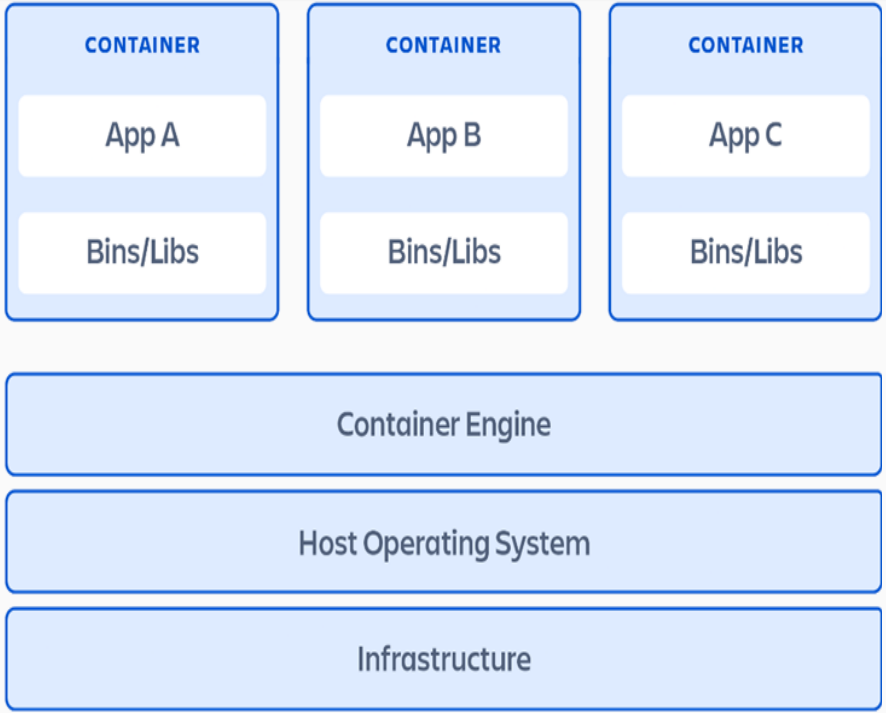


Containerization



Virtual Machines vs Containers





Virtual Machines vs Containers

Virtual Machines virtualize an entire machine down to the hardware layers

Containers only virtualize software layers above the operating system level.

Size in GBs

Size in MBs

Spins up slower than containers

Spins up much faster reuses the host kernel

Docker Concepts

Docker

An open platform for developing, shipping and running applications using containers.

Another definition:

“Docker is a tool that packages software into containers to run reliably in any environment.”

Container

Light weight, standalone, executable package that includes:

Code

Runtime

System tools

Libraries

Settings

Image

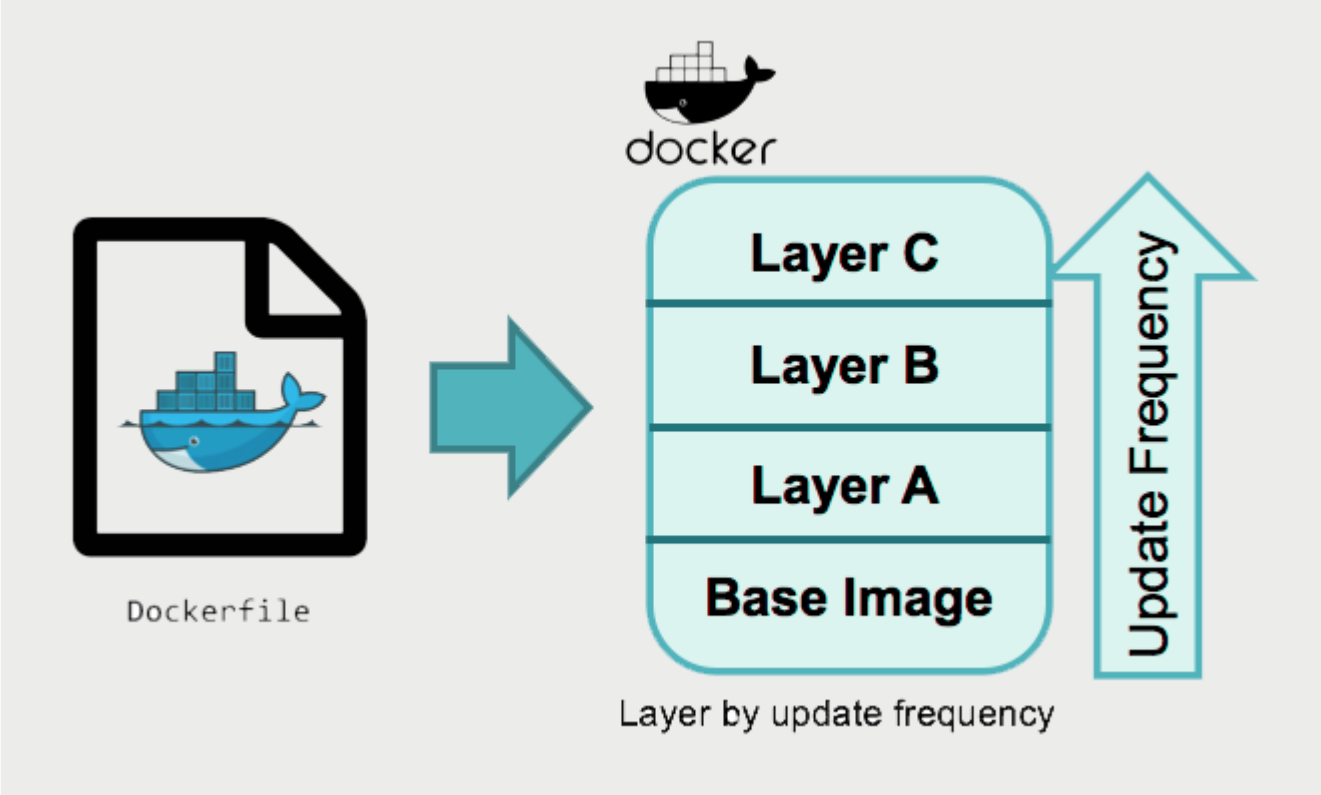
A container image is a standardized package that includes all of the files, binaries, libraries, and configurations to run a container

Snapshot of filesystem and parameters

Template that will be used for the container

.

Image layers



DockerFile

Format:

```
# Comment
INSTRUCTION arguments
```

Dockerfile

```
# Use Python base image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy files
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

# Expose port and run the app
EXPOSE 5000
CMD ["python", "app.py"]
```

Core Docker Concepts

Image: A blueprint for containers (like a class in OOP)

Container: A running instance of an image (like an object)

Dockerfile: Instructions to build a Docker image

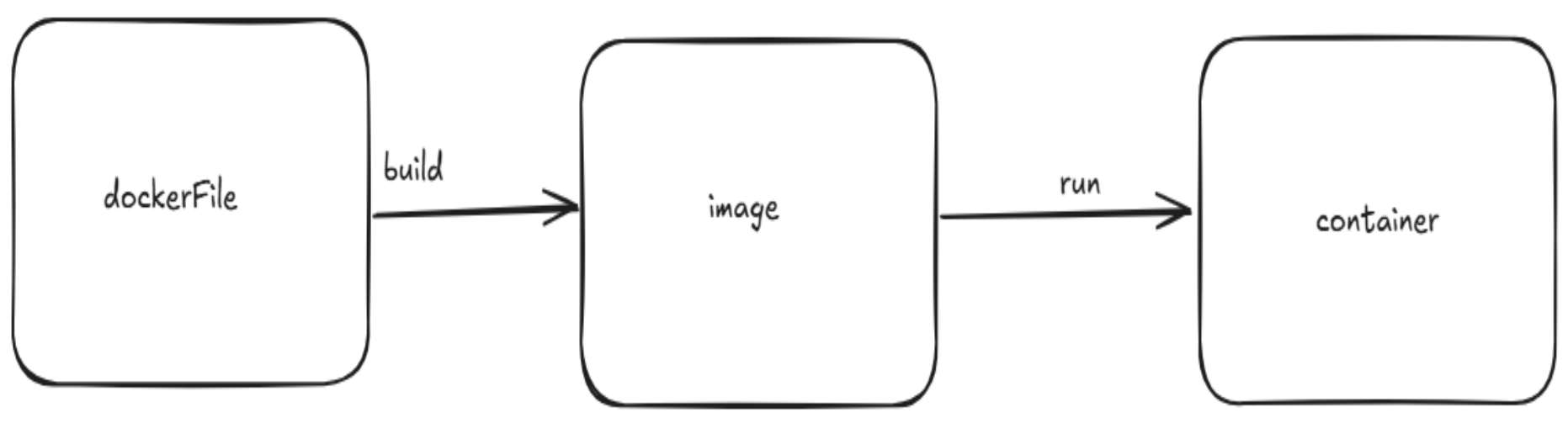
Volume: Way to persist data outside the container

Docker Compose: Tool for defining and running multi-container apps

Registry: A storage for Docker images

Compose: Define multiple containers

Docker Commands


dockerFile
build
image
run
container

IMAGES

Docker images are a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.

Build an Image from a Dockerfile
```
docker build -t <image_name> .
```

Build an Image from a Dockerfile without the cache
```
docker build -t <image_name> . —no-cache
```

List local images
```
docker images
```

Delete an Image
```
docker rmi <image_name>
```

Remove all unused images
```
docker image prune
```

DOCKER HUB

Docker Hub is a service provided by Docker for finding and sharing container images with your team. Learn more and find images at https://hub.docker.com

Login into Docker
```
docker login -u <username>
```

Publish an image to Docker Hub
```
docker push <username>/<image_name>
```

Search Hub for an image
```
docker search <image_name>
```

Pull an image from a Docker Hub
```
docker pull <image_name>
```

CONTAINERS

A container is a runtime instance of a docker image. A container will always run the same, regardless of the infrastructure. Containers isolate software from its environment and ensure that it works uniformly despite differences for instance between development and staging.

Create and run a container from an image, with a custom name:
```
docker run --name <container_name> <image_name>
```

Run a container with and publish a container's port(s) to the host.
```
docker run -p <host_port>:<container_port> <image_name>
```

Run a container in the background
```
docker run -d <image_name>
```

Start or stop an existing container:
```
docker start|stop <container_name> (or <container-id>)
```

Remove a stopped container:
```
docker rm <container_name>
```

Open a shell inside a running container:
```
docker exec -it <container_name> sh
```

Fetch and follow the logs of a container:
```
docker logs -f <container_name>
```

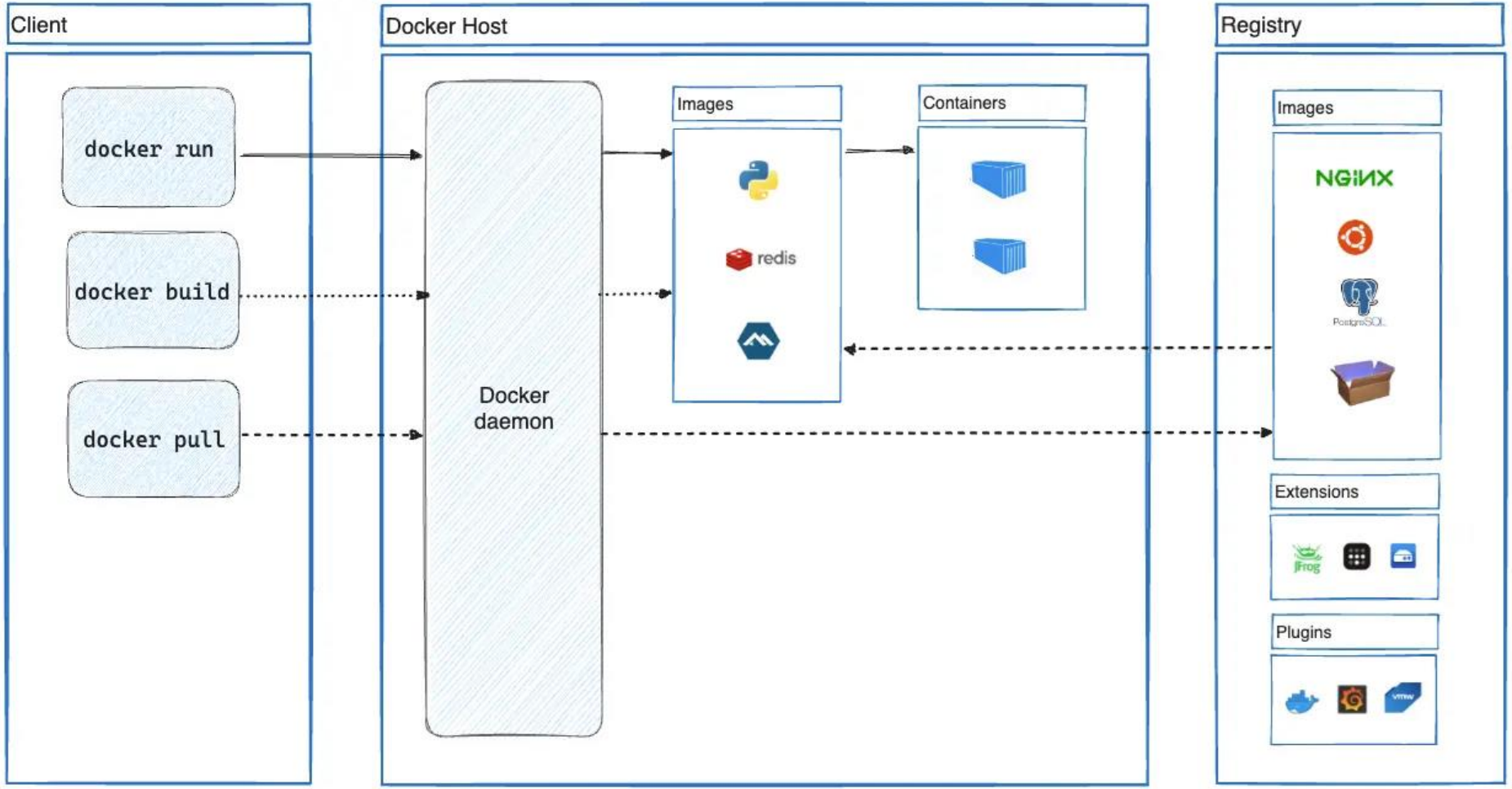
To inspect a running container:
```
docker inspect <container_name> (or <container_id>)
```

To list currently running containers:
```
docker ps
```

List all docker containers (running and stopped):
```
docker ps --all
```

View resource usage stats
```
docker container stats
```

Wrap it up


Client
docker run
docker build
docker pull
Docker Host
Docker daemon
Images
redis
Containers
Registry
Images
NGINX
PostgreSQL
Extensions
jfrog
Plugins
vmw

Thanks!